

CURRICULUM PORTFOLIO

Python Basics

Week 1 — Beginner Curriculum

ADDIE Framework | Bloom's Taxonomy | Full Lesson Plans | MCQ Assessments |
Differentiation Guide

ATHIRA A

Instructional Designer | Curriculum Developer | AI in Education

github.com/Athiraajith91 | athiraajith91@gmail.com

Subject	Python Programming for Beginners
Target Audience	Ages 10-14 — Middle School — Zero prior coding experience
Duration	1 Week — 5 Days — 45 to 60 minutes per session
Tools	Replit / IDLE / Code.org Python Lab — browser-based, no installation
Delivery Mode	Online 1:1 and small group — international learners
ID Framework	ADDIE Model + Bloom's Taxonomy
Date	2026

Day 1	Introduction to Python and Printing Output
Day 2	Variables and Data Types
Day 3	Input and Basic Expressions
Day 4	Conditional Statements — if/else
Day 5	Project Day — Student Choice Capstone

LEARNER PROFILE

Understanding the learner is the foundation of every instructional design decision. This profile was built from 5+ years of direct teaching experience with beginner coders aged 10-16 across Cuemath, Codeyoung, Turito, and Codingal.

Age Range	10 to 14 years — Middle School learners (Grades 5 to 8)
Prior Knowledge	None — absolute beginners with zero programming experience
Learning Environment	Online 1:1 or small group — international students
Attention Span	30 to 45 minutes focused coding — needs activity switches every 10-15 min
Learning Style	Learn by doing — needs immediate hands-on practice after each concept
Motivation	Curious — motivated by real programs they can run and show to others
Confidence Level	Low at start — easily discouraged by errors — needs frequent wins
Digital Comfort	High comfort with devices — low comfort with text editors and IDEs
Device	Laptop or desktop with stable internet — browser-based tools only

Key Learner Needs

- Start with real-life examples before showing code
- Give small, quick wins to build confidence
- Normalize errors — mistakes are part of learning
- Provide real, shareable projects learners can show others
- Allow personal choices so projects feel meaningful to them

CURRICULUM LEARNING OBJECTIVES

All objectives are written using Bloom's Taxonomy action verbs and are measurable through assessments embedded in each day's lesson plan.

Bloom's Taxonomy Alignment

1 — Remember	Recall Python syntax for print(), variables, input(), and if/else statements
2 — Understand	Explain how each Python concept works using real-world analogies
3 — Apply	Write working Python programs using all four core concepts independently
4 — Analyse	Identify and debug common Python syntax and logic errors independently
5 — Create	Design and build a complete mini-project combining all concepts learned

Day-by-Day Objective Map

Day 1	Learner can write Python programs using print() to display text, numbers, and expressions
Day 2	Learner can create variables of type string, integer, and float and print their values
Day 3	Learner can build interactive programs using input(), type conversion, and arithmetic
Day 4	Learner can use if/else and elif to make decisions based on user input
Day 5	Learner can independently design and build a working mini-project using all four concepts

Success Criteria

A learner has successfully completed this curriculum when they can:

- Write a program that takes user input and prints a personalised message
- Store values in variables and perform calculations correctly
- Use if/elif/else to make the program respond differently based on input
- Read an error message and identify what went wrong without teacher help
- Explain their Day 5 project — what it does and how the logic works

ADDIE PROCESS DOCUMENTATION

The ADDIE instructional design model was applied throughout the development of this curriculum. Each phase is documented below with the specific decisions made.

ANALYSE

Identified learner profile from 5+ years of direct teaching experience. Key findings: learners disengage when syntax precedes context, errors cause confidence loss, and visible program outputs are the strongest motivator. Delivery constraint: online 1:1 with 45-60 minute window requiring activity switches every 10-15 minutes.

DESIGN

Designed 5-day scope and sequence: print → variables → input → conditions → project. Each day follows an identical lesson architecture for cognitive consistency — warm-up, concept, guided practice, independent practice, challenge, exit ticket, homework. Applied Bloom's Taxonomy: Remember (Day 1) through Create (Day 5).

DEVELOP

Independently developed all lesson materials — lesson plans with teacher scripts, tested code examples, guided and independent practice tasks,

	MCQ quizzes with answer keys, exit tickets, homework tasks, and Day 5 capstone project menu with four choice-based options.
IMPLEMENT	Designed for platform-agnostic online delivery using Replit, IDLE, or Code.org — no software installation required. Teacher notes are embedded in every lesson. Code examples scaffolded from teacher-modelled to student-independent. Pacing guidance for 45-minute and 60-minute variants.
EVALUATE	Formative: exit tickets and MCQ quizzes in every day. Summative: Day 5 capstone project evaluated on correct concept application, logical structure, and student explanation during showcase. Homework tasks provide spaced practice between sessions.

CURRICULUM OUTLINE

Day 1	Introduction to Python and Printing Output — <code>print()</code>, text, numbers, errors
Day 2	Variables and Data Types — string, int, float, variable naming rules
Day 3	Input and Basic Expressions — <code>input()</code> , arithmetic operators, type conversion
Day 4	Conditional Statements — if, elif, else, comparison operators
Day 5	Project Day — Age Calculator, Number Comparison, Login System, Chat Responder

DAY 1-INTRODUCTION TO PYTHON AND PRINTING OUTPUT

Lesson Title: Introduction to Python & Printing Output

Grade Level: Beginner (Age 10-14 / Coding Beginners)

Duration: 45–60 minutes

1. Learning Objectives

By the end of this lesson, students will be able to:

- Describe what a programming language does.
- Understand what Python is and where it is used.
- Write their first Python program using the `print ()` function.
- Print text, numbers, and simple expressions in Python.
- Identify common printing errors (missing quotes, parentheses).

2. Materials Needed

- Laptop or desktop with internet access
- Python IDE (Replit / IDLE / Code.org Python Lab)

3. Lesson Introduction (10 minutes)

"Imagine if your computer could understand your instructions exactly like a smart friend.

Python is a language that allows us to talk to the computer!"

Explain:

- Python is a programming language used for apps, websites, AI, and automation.
- It is easy to read and great for beginners.
- Today, students will create their first Python program.

Real-life examples of Python usage:

- Instagram backend
- Google search algorithms (parts)
- Robotics & automation
- Data science and AI models

4. Direct Instruction (15 minutes)

4.1 What is print ()?

- Python uses the print () function to show output on the screen.
- Think of it as asking the computer to "say something".

Syntax:

```
print ("Your message here")
```

4.2 How to Print Text

- Text must be inside quotes.

Example:

```
print ("Hello, Python!")
```

Output:

```
Hello, Python!
```

4.3 How to Print Numbers

- Numbers do not need quotes.

Example:

```
print (25)
```

Output:

```
25
```

4.4 Printing Multiple Items

```
print ("My age is", 12)
```

output:

My age is 12

4.5 Why Quotes Matter

Explain the difference:

```
print ("20") # Text
```

```
print (20) # Number
```

4.6 Common Errors

Missing Quotes:

```
print (Hello)
```

Error: NameError

Missing Parenthesis:

```
print "Hello" Error: SyntaxError
```

5. Guided Practice (10 minutes)

Students try the following with teacher support:

- ❖ Print their name: `print ("My name is Ann")`
- ❖ Print their favourite food: `print ("I love biryani")`
- ❖ Print a number: `print (2025)`
- ❖ Print a sentence with text + number: `print ("I am", 20, "years old")`

The teacher checks and corrects errors.

6. Independent Practice (10 minutes)

Students complete small tasks:

- Task 1: Print 3 sentences about yourself.
- Task 2: Print your favourite hobby.
- Task 3: Print any number between 1–100.
- Task 4: Print this exact output:
Python is fun!

7. Mini Project (5 minutes)

Create a Personal Introduction Program. Students must include:

- Name
- Age
- One favourite thing
- One dream for the future
- A number of their choice

Sample Output:

```
My name is Rahul.
```

```
I am 12 years old.
```

```
I love football. My dream is to become a scientist.
```

```
My favourite number is 18.
```

8. Assessment / Exit Ticket (5 minutes)

Ask students to answer:

1. What does the `print ()` function do?
2. Why do we use quotes?
3. Write a correct Python print statement for displaying “ My best friend is Ann, and she is 10 years old”

9. Homework

Create a Python script with 5 print statements describing:

- ❖ Your best friend
- ❖ A place you want to visit
- ❖ A number fact
- ❖ A motivational quote
- ❖ Anything funny

Differentiation — Day 1

Fast Finishers	Struggling Learners	Common Errors to Watch
• Print a formatted receipt with items and prices • Print a pattern using multiple print statements	• Provide a copy-and-modify template — learner changes values only • Focus on one task at a time — do not show all tasks at once • Allow the teacher to type while the learner reads and explains	• <code>print (Hello)</code> — <code>NameError</code> — missing quotes around text • <code>print ('Hello'</code> — <code>SyntaxError</code> — missing closing parenthesis • Putting quotes around numbers unnecessarily — <code>print ('25')</code>

MCQ Quiz + Answer Key

1. What does the print () function do in Python?

- a) Adds two numbers
- b) Shows output on the screen
- c) Deletes text
- d) Saves the program

Ans: b) Shows output on the screen

2. Which of the following is the correct way to print text?

- a) print (Hello)
- b) print ("Hello")
- c) print ('Hello')
- d) print (Hello')

Ans: b) print("Hello")

3. Which of these does NOT need quotes?

- a) "Hello"
- b) "Python"
- c) 25
- d) "2025"

Ans: c) 25

4. What error occurs when you forget quotes around text?

a) ZeroDivisionError

b) NameError

c) TypeError

d) ValueError

Ans: b) NameError

5. Which command will print the number 10?

a) print ("10")

b) print (10)

c) print(ten)

d) print("ten")

Ans: b) print (10)

Teacher Notes / Tips

- Encourage students to try different variations of print.
- Remind them: text → quotes, numbers → no quotes.
- Ask them to read error messages — learning to debug is important.
- Celebrate their first successful program.

DAY 2 – VARIABLES & DATA TYPES

Lesson Title: Variables & Data Types

Duration: 45–60 minutes

Materials: Laptop, Replit/Compiler

Learning Objectives

Students will:

- Define a variable
- Create variables
- Identify string, int, float
- Print values
- Write simple programs

Lesson Flow

1. Warm-Up – 5 min

Ask: “Where do we store things in real life?”

Example responses: water in a bottle, food in a lunch box, money in a wallet.

Explain:

“Just like we use containers to store different things in real life, Python also needs a place to store information.

These storage places are called variables.”

Tell learners:

“Variables help the computer remember information so we can use it later in our code — just like how we remember what's inside our bottle or lunch box.”

2. Introduction to Variables – 10 min

- A variable means a name that holds a value.
- `variable_name = value`
- Whenever we use the equal sign `=`, we are storing something inside the variable.
- Rules in naming a variable:
 - ❖ A variable name CANNOT start with a number.
 - ❖ A variable name can only contain letters, numbers, and underscores.
 - ❖ No spaces allowed in variable names.
 - ❖ Variable names are case-sensitive.
 - ❖ A variable cannot use Python keywords.
 - ❖ Choose meaningful names

Examples:

```
name = "John"  
age = 12
```

4. Data Types – 10 min

Data type tells the computer what kind of value you are working with. Think of them like different boxes to store different kinds of things.

- String – text, used for names, messages, sentences, mobile numbers, etc.
- Integer – whole number, used for counting, math operations, age, marks.
- Float – decimal, used for money, measurements, and calculations with fractions.

Examples:

```
city = "Mumbai" → string  
height = 160 → integer  
temperature = 36.6 → float
```

4. Hands-On Coding – 15 min

Students create: Let them create 3 variables and store values in them, and then display.

```
fruit = "Apple"  
quantity = 5  
price = 12.50  
print(fruit)  
print(quantity)  
print(price)
```

Output:

```
Apple  
5  
12.50
```

5. Challenge Activity – 10 min

Students create a personal info program using variables, which should include name, age, class, School name, parents' names, and addresses as variables, store values in them, and display them.

6. Assessment (Exit Ticket)

- What is a variable?
- Example of a string variable
- Create a float variable
- Predict output

```
x = 12  
y = 10  
print (y)  
print (x)
```

7. Homework

Create a Python program that stores the following information using variables:

Your favorite movie

The year it was released

The main actor's name

The rating you would give (out of 10, using a float)

Then print all four values.

Differentiation — Day 2

Fast Finishers	Struggling Learners	Common Errors to Watch
• Build a program storing 5 facts and printing a bio • Explore type () function to check variable data types • Create variables of each type and test which arithmetic works	• Provide variable name and type — learner fills in the value only • Use physical analogy — variable is a labelled box with one item inside • Focus on string and integer only — skip float on first pass	• Starting variable name with a number — 1name = 'Ann' causes SyntaxError • Adding quotes to number variables — age = '12' stores string not integer • Spaces in variable names — first name = 'Ann' causes SyntaxError

MCQ QUIZ (10 Questions)

Q1. What is Python?

- a. A type of snake
- b. A programming language
- c. A computer
- d. None

Answer: b

Q2. Which function prints output?

- a. show ()
- b. output ()
- c. print ()
- d. display ()

Answer: c

Q3. Which is a string?

- a. 12
- b. 4.5
- c. "Hello"
- d. -7

Answer: c

Q4. Which is an integer?

- a. "14"
- b. 3.14
- c. 14
- d. "3.14"

Answer: c

Q5. Which of these is a valid variable name?

- a. 1name
- b. my name
- c. my_name

- d. my-name

Answer: c

Q6. What will this print?

```
name = "Alice"
```

```
print(name)
```

- a. "Alice"
- b. name
- c. Alice
- d. Error

Answer: c

Q7. What does this code do?

```
age = 10
```

```
print(age)
```

- a. Prints 10
- b. Prints age
- c. Error
- d. Prints nothing

Answer: a

Q8. float refers to:

- a. Whole numbers
- b. Decimal numbers
- c. Text
- d. True or False

Answer: b

Q9. Which is a correct variable?

- a. 1name = "Ann"
- b. @age = 10
- c. name = "Ann"
- d. #city = "Delhi"

Answer: c

Q10. If $x = 5$ and $y = 3$, what is $x + y$?

- a. 8
- b. 2
- c. 53
- d. Error

Answer: a

Day 3 – Input & Basic Expressions

Lesson Title: Input & Basic Expressions

Duration: 45–60 minutes

Materials: Laptop, Replit/Compiler

Learning Objectives

By the end of this lesson, students will be able to:

- Use the input () function to take user input.
- Perform basic arithmetic operations (+, −, *, /, //, %, **).
- Convert data types (e.g., int (), float (), str ()).
- Build small interactive programs using input and expressions.

1. Warm-Up – 5 min

Ask students: “How do websites or apps know your name or age?”

How are the apps and website doing conversions like degrees Celsius to Fahrenheit?”

Explain: Programs ask users for information using the input () function. And also, it uses basic operators to perform these conversions

2. Introduction to Input – 10 min

Concept: input () lets the user type something during the program.

It always returns a string.

Example:

```
name = input ("Enter your name: ")  
print ("Hello", name)
```

- The program shows the message: *Enter your name:*
- The user types something
- The value they typed gets stored in the variable name
- print () displays the message with the name

Output:

Hello, Ann

Discuss how input makes programs interactive.

3. Basic Expressions & Type Conversion – 10 min

Introduce operators:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Floor Division (//)
- Modulus (%)
- Exponent (**)

Explain type conversion: int (), float (), str ()

Example:

```
age = int (input ("Enter your age: "))  
years = age + 5  
print ("You will be", years, "in 5 years.")
```

4. Hands-On Coding – 15 min

Students create a small program using input and operators:

```
num1 = float (input ("Enter first number: "))  
num2 = float (input ("Enter second number: "))  
print ("Sum =", num1 + num2)  
print ("Difference =", num1 - num2)  
print ("Product =", num1 * num2)  
print ("Division =", num1 / num2)
```

The teacher checks if students used the correct type conversion.

5. Challenge Activity – 10 min

Task: Birthday Age Predictor

Students enter their birth year → program calculates age.

Uses: input(), int(), subtraction

```
birth_year = int(input("Enter your birth year: "))  
current_year = 2025  
age = current_year - birth_year  
print("You are", age, "years old.")
```

6. Assessment (Exit Ticket)

1. What does the input () function do?
2. Why do we use int () or float ()?
3. What is the output of: print (10 % 3)?

4. Predict the output of this program:

```
x = int(input("Enter number: "))  
print(x * 2)
```

7. Homework

Create a program that asks the user for two numbers and prints:

- The smallest number
- Their average
- The larger number

Use input, operators, and type conversion.

Differentiation — Day 3

Fast Finishers	Struggling Learners	Common Errors to Watch
• Build a BMI calculator using input, conversion, and arithmetic • Create a temperature converter — Celsius to Fahrenheit • Explore what happens when user types letters instead of numbers	• Pre-write the input and conversion lines — learner writes calculation only • Use physical analogy — input is asking someone a question and writing their answer • Focus on addition only before introducing all operators	• Forgetting to convert input — age = input() stores string not integer • Using x instead of * for multiplication — common math habit • Division without conversion giving unexpected decimal results

MCQ QUIZ (10 Questions)

1. What does the input () function return by default?

- a. Integer
- b. Float
- c. String
- d. Boolean

Answer: c

2. Which function is used to convert a string to an integer?

- a. str()
- b. float()
- c. int()
- d. input ()

Answer: c

3. What is the output of: int ("5") + int ("3")?

- a. 53
- b. 8
- c. 5+3
- d. Error

Answer: b

4. Which operator performs exponentiation in Python?

- a. ^
- b. **
- c. *
- d. //

Answer: b

5. What will 10 // 3 return?

- a. 3.33
- b. 3
- c. 4
- d. 0

Answer: b

6. Which of the following is correct for taking a number as input?

- a. `num = input (int ("Enter a number: "))`
- b. `num = int (input ("Enter a number: "))`
- c. `num = input ("Enter a number:", int)`
- d. `num = number (input ())`

Answer: b

7. What will be the output of: `print (7 % 3)`?

- a. 1
- b. 2
- c. 3
- d. 4

Answer: a

8. Which of these is NOT an arithmetic operator?

- a. +
- b. %
- c. //
- d. &&

Answer: d

9. What happens if you add a string and an integer?

- a. They add normally
- b. Python converts automatically
- c. Error: Type mismatch
- d. It prints them together

Answer: c

10. Which statement correctly prints the user's name?

- a. `print ("Hello" + name)`
- b. `print ("Hello", name)`
- c. Both a and b
- d. None

Answer: c

Day 4 – Conditional Statements (if/else)

Lesson Title: Conditional Statements (if/else)

Duration: 45–60 minutes

Materials: Laptop, Replit/Compiler

Learning Objectives

By the end of this lesson, students will be able to:

- Understand what a condition is
- Compare numbers and strings using comparison operators
- Use if and else to make decisions in programs

Lesson Flow

1. Warm-Up – 5 min

Ask students:

“Have you ever said: ‘If it rains, I will take an umbrella’?”

Explain:

- This is an example of a **condition** in real life.
- Computers also use **if/else** to make decisions.

Example for kids:

- **If** you finish homework → **you can play**.
- **Else** → **you cannot play**.

2. Introduction to Conditions – 10 min

Explain:

A **condition** checks whether something is **True** or **False**.

Introduce **comparison operators**:

- == equal to
- != not equal to
- > greater than
- < less than
- >= greater than or equal
- <= less than or equal

Example:

```
Print (5 > 3) # True
```

```
Print ("A" == "A") # True
```

```
Print (10 != 7) # True
```

3. Using if/else – 10 min

Explain structure:

```
if condition:
```

```
# code runs if True
```

```
else:
```

```
# code runs if False
```

Explain **indentation**:

- Python uses spaces to show which code belongs to if/else.
- Indent = 4 spaces or 1 tab.

Explain **elif**:

- If we have multiple conditions to check before else, then we can make use of elif

```
if condition:
# code runs if this condition is True

elif:
# code runs if this condition is True

else:
# code runs if all the above conditions are False
```

Example:

```
age = 15

if age >= 18:
print ("You can vote.")
else:
print ("You cannot vote yet.")
```

4. Hands-On Coding – 15 min

Students practice the following programs:

a) Grading System

```
marks = int (input ("Enter your marks: "))

if marks >= 90:
print ("Grade A")
else:
print ("Keep trying!")
```

b) Login Simulation

```
username = input ("Enter username: ")

if username == "admin":
print ("Login successful")
```

```
else:  
    print ("Wrong username")
```

5. Challenge Activity – 10 min

Task:

Create a program that checks whether a number is:

- Positive
- Negative
- Zero

Students must use if, elif, and else.

Assessment (Exit Ticket)

1. What is a condition?
2. Write an example of an if/else statement.
3. What is the output of:

```
if 5 > 10:  
    print("Yes")  
else:  
    print("No")
```

4. What is the usage of ==

Homework

Create a Python program using if/elif/else to display the day according to numbers from 1-7

Ask the user to enter a number (1–7).

1 → Monday, 2 → Tuesday, ..., 7 → Sunday

Else → "Invalid day number"

Uses multiple elif.

Differentiation — Day 4

Fast Finishers	Struggling Learners	Common Errors to Watch
• Build a grade calculator with A, B, C, D, F using elif chain • Create a number guessing game using nested conditions • Add password check to login simulation with multiple attempts	• Provide if/else skeleton — learner fills in condition and output only • Use a flowchart diagram to map logic before writing code • Focus on if and else only — skip elif on first pass	• Using = instead of == for comparison — assigns value instead of comparing • Forgetting the colon after if condition — causes SyntaxError • Indentation errors — Python blocks require exactly 4 spaces or one tab consistently

10 MCQs for Conditional Statements (if/else)

1. Which symbol is used for equality comparison in Python?

- a. =
- b. ==
- c. ===
- d. !=

Answer: b

2. What will this code print?

```
if 10 > 5:
```

```
    print("Yes")
```

```
else:
```

```
    print("No")
```

- a. Yes
- b. No
- c. Error
- d. Nothing

Answer: a

3. Which of the following is a valid if statement?

- a. if x > 3:
- b. if (x > 3)
- c. if x > 3 then
- d. if x > 3;

Answer: a

4. What does the else part do?

- a. Runs only if the condition is true
- b. Runs only if the condition is false
- c. Runs only when there is an error
- d. Runs before the if condition

Answer: b

5. What is the output of:

```
if "A" == "a":
```

```
    print("Match")
```

```
else:
```

```
    print ("No Match")
```

- a. Match
- b. No Match
- c. Error
- d. a

Answer: b

6. Which operator checks “not equal to”?

- a. =!
- b. !=
- c. !=
- d. <>

Answer: c

7. Identify the correct structure:

- a. if condition: print("Hello")
- b. if condition: print("Hello")
- c. if condition: print("Hello")
- d. if (condition): print("Hello")

Answer: c

8. What will this code print?

```
age = 12
```

```
if age >= 18:
```

```
    print("Adult")
```

```
else:
```

```
    print("Child")
```

- a. Adult
- b. Child
- c. Nothing
- d. Error

Answer: b

9. What type of value does a condition return?

- a. int
- b. string
- c. boolean (True/False)
- d. float

Answer: c

10. Which statement is true about if/else?

- a. else must always come before if
- b. if can exist without else
- c. A program can only have one if statement
- d. if/else only works with numbers

Answer: b

Day 5 – Project Day

Theme: Apply all concepts learned from Days 1–4
(Input, Variables, Data Types, Expressions, if/else)

Lesson Flow

1. Warm-Up – 5 min

Ask students: “What kind of programs do you want to build in real life?”

Explain that today they will choose and build their own mini-project.

2. Project Options – 5 min

Students choose one of the following projects:

Mood-Based Music Recommendation

Ask the user about their mood and give a music suggestion.

Concepts Used:

- ❖ input
- ❖ string comparison
- ❖ if/elif/else

Sample Logic:

```
mood = input ("How are you feeling today? (happy/sad/angry/tired): ")
if mood == "happy":
    print ("Play: Happy Dance Playlist!")
elif mood == "sad":
    print ("Play: Calm & Comfort Songs")
elif mood == "angry":
```

```
print ("Play: Relaxing Instrumentals")

elif mood == "tired":

    print ("Play: Soft Sleep Music")

else:

    print ("Mood not recognized!")
```

Food Calorie Checker

Ask the user which food they ate → tell calories.

Concepts Used:

- ❖ input
- ❖ variables
- ❖ if/elif/else

Sample Logic:

```
food = input ("Enter a food (apple/banana/pizza): ")

if food == "apple":

    print ("Approx calories: 95")

elif food == "banana":

    print ("Approx calories: 105")

elif food == "pizza":

    print ("Approx calories: 285")

else:

    print ("Food not found")
```

Travel Ticket Price Checker

Ask destination → show ticket price.

Concepts Used:

- ❖ input
- ❖ if/elif/else

Sample Logic:

```
place = input ("Enter destination (Delhi/Mumbai/Goa): ")
if place == "Delhi":
    print ("Ticket Price: ₹3500")
elif place == "Mumbai":
    print ("Ticket Price: ₹3000")
elif place == "Goa":
    print ("Ticket Price: ₹2500")
else:
    print ("Destination not available.")
```

Movie Category Selector

Ask the user their age → suggest movie type.

Concepts Used:

- ❖ input(int)
- ❖ if/elif/else

Sample Logic:

```
age = int (input ("Enter your age: "))  
if age < 7:  
    print ("You can watch: Cartoon movies")  
elif age < 13:  
    print ("You can watch: Kids Adventure movies")  
elif age < 18:  
    print ("You can watch: Teen Movies")  
else:  
    print ("You can watch: All movie categories!")
```

3. Work Time – 20 min

Students code their selected project individually or in pairs. The teacher helps with debugging and logic.

4. Showcase – 10 min

Students run their program and explain:

- Which project did they choose
- What inputs does it take
- How the logic works

5. Learning Outcome

By the end of Day 5, students can:

- Use input correctly
- Declare variables
- Perform calculations

- Make decisions using if/else
- Combine all these skills to create a working mini-project

6. Assessment (Exit Ticket)

- What project did you choose?
- Which Python concept was hardest for you?
- What would you add if you had more time?

7. Homework

Improve today's project by adding one extra feature.

Examples:

- Mood-Based Music Recommendation.
Extra Feature: Suggest 2 songs/playlists instead of one.
- Food Calorie Checker.
Extra Feature: Ask if they want another item, display total calories.
- Travel Ticket Price Checker.
Extra Feature: Add a discount for students.
- Movie Category Selector.
Extra Feature: Recommend 1 movie from the chosen category.

Differentiation — Day 5

Fast Finishers	Struggling Learners	Common Errors to Watch
• Add another feature to their chosen project • Combine two project	• Provide project structure as comments — learner fills in each	• Combining concepts incorrectly — using string where integer is

options into one program • Add input validation — what if the user types the wrong thing	section • Reduce to the simplest version — Mood Music with just happy/sad (2 options only) • Work through the project together — teacher types the learner's instructions	needed • Forgetting to convert input when using in arithmetic or comparison • Logic errors — condition correct syntactically, but gives wrong output
---	---	--

TEACHER NOTES AND GENERAL TIPS

General Classroom Tips — Online Delivery

- Encourage learners to try different variations — exploring is part of coding
- Celebrate every working program regardless of complexity — the first print statement matters
- Ask learners to read error messages — learning to debug is a critical skill
- Never compare learners — every learner progresses at their own pace in coding
- Use errors as teaching moments — show that even experienced programmers make the same mistakes
- Let fast finishers explain to struggling learners — peer teaching deepens understanding for both
- Always end with a win — if a learner struggles, simplify the task so they finish on a positive note

Pacing Guidance

- 45-minute session: Skip challenge activity — use guided + independent practice only
- 60-minute session: Include full lesson flow, including challenge activity and project
- If learner finishes early: Assign fast finisher extension from the differentiation table
- If learner struggles: Switch to simplified scaffold from differentiation table immediately